BERT, by hand

Worksheet #38181 · sentence A “red”, sentence B “cat” · n = 5, H = 4, A = 2, d_k = 2, d_ff = 4, one encoder layer plus the MLM and NSP heads · from BERT by Hand

Name _____ Date _____

Write one number in each empty box, rounded to 2 decimals, and use your rounded answers in the steps after. Shaded tables are given to you. A basic calculator helps for √, e^x, ln and tanh. Every step has a hint at the bottom, and the answer key is on the last pages.

Step 1: Pack the pair and look up the IDs

BERT reads two sentences at once, glued with special pieces: **[CLS]** first, **[SEP]** after each sentence. Position 1 (“red”) was picked for masking and swapped for **[MASK]**. The pair's label is **NotNext**.

position	0	1	2	3	4
original	[CLS]	red	[SEP]	cat	[SEP]
model reads	[CLS]	[MASK]	[SEP]	cat	[SEP]

Token table: id, piece, embedding row

	d0	d1	d2	d3
0 [PAD]	0	1	-1	-1
1 [CLS]	-1	-1	0	0
2 [SEP]	-1	-1	-1	-1
3 [MASK]	1	1	-1	1
4 cat	1	1	0	1
5 red	0	0	0	-1
6 runs	-1	-1	0	1

input IDs, shape (5,)

[CLS]	[MASK]	[SEP]	cat	[SEP]

segment IDs

[CLS]	[MASK]	[SEP]	cat	[SEP]

Hint. Look up what the model *reads*, so position 1 is the ID of [MASK], not of “red”. Segment ID is 0 from [CLS] through the first [SEP], and 1 after it.

Step 2: Token embeddings

`E_tok = the token-table row of each input ID · shape 5 × 4`

E_tok (5 × 4)

	d0	d1	d2	d3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

Hint. Copy the whole row. Row 1 is the row of [MASK]: the model never sees “red” here.

Step 3: Segment and position embeddings

`E_seg = the segment-table row of each segment ID · E_pos = row p of the position table`

Segment table

	d0	d1	d2	d3
0 (A)	-1	0	-1	-1
1 (B)	0	1	-1	-1

Position table (learned, not sin/cos)

	d0	d1	d2	d3
pos 0	0	0	1	-1
pos 1	-1	1	-1	1
pos 2	1	-1	0	-1
pos 3	-1	0	1	1
pos 4	-1	1	0	0

E_seg

	d0	d1	d2	d3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

E_pos

	d0	d1	d2	d3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

> Hint. Two more lookups, no arithmetic. BERT learns its position rows like any other weight, so you copy row p instead of computing sines.

Step 4: Add the three lookups

`X_sum = E_tok + E_seg + E_pos · shape 5 × 4`

X_sum

	d0	d1	d2	d3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

> Hint. Add the numbers in the same row and column of all three tables: X_sum[0 [CLS], d0] = E_tok + E_seg + E_pos at [0 [CLS], d0]. There is no √d scaling in BERT.

Step 5: Embedding LayerNorm

`X0 = (X_sum - row mean) ÷ row sd`

mean

0 [CLS]	
1 [MASK]	
2 [SEP]	
3 cat	
4 [SEP]	

sd

0 [CLS]	
1 [MASK]	
2 [SEP]	
3 cat	
4 [SEP]	

X0 (5 × 4)

	d0	d1	d2	d3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

Hint. The released code normalises the sum before the first layer. Row mean = (sum of the 4 numbers) ÷ 4. Row sd = √(average of (value − mean)²), dividing by 4. Then (value − row mean) ÷ row sd. γ = 1 and β = 0 here, so nothing else changes. Check: each row adds up to about 0.

Step 6: Head 1: queries, keys and values

`Q = X0 · W_Q, K = X0 · W_K, V = X0 · W_V · (5 × 4) · (4 × 2) = 5 × 2`

W_Q

	0	1
d0	0	0
d1	-1	0
d2	0	0
d3	0	-1

W_K

	0	1
d0	1	0
d1	-1	-1
d2	0	0
d3	0	0

W_V

	0	1
d0	1	0
d1	0	-1
d2	1	0
d3	0	0

Q

	0	1
0 [CLS]		
1 [MASK]		
2 [SEP]		
3 cat		
4 [SEP]		

K

	0	1
0 [CLS]		
1 [MASK]		
2 [SEP]		
3 cat		
4 [SEP]		

V

	0	1
0 [CLS]		
1 [MASK]		
2 [SEP]		
3 cat		
4 [SEP]		

Hint. Each entry is one row of X0 dotted with one column of the weights: Q[0 [CLS], 0] = X0[0 [CLS], d0]×0 + X0[0 [CLS], d1]×(−1) + X0[0 [CLS], d2]×0 + X0[0 [CLS], d3]×0. The real model also adds a bias here; this sheet leaves it out.

Step 7: Head 1: scores $S = Q \cdot K^T$

`S[r → c] = Q[r] · K[c] · shape 5 × 5`

S (rows: query, columns: key)

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]					
1 [MASK]					
2 [SEP]					
3 cat					
4 [SEP]					

Hint. S[0 [CLS] → 1 [MASK]] = Q[0 [CLS], 0]×K[1 [MASK], 0] + Q[0 [CLS], 1]×K[1 [MASK], 1]. No mask: every piece may look at every other, in both sentences, in both directions.

Step 8: Head 1: scale by √d_k

`S ÷ √2 = S ÷ 1.41`

S ÷ 1.41

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]					
1 [MASK]					
2 [SEP]					
3 cat					
4 [SEP]					

Hint. Divide every score by 1.41.

Step 9: Head 1: softmax, row by row

`A[r, c] = e^x[r, c] ÷ (sum of e^x across row r)`

e^x of each scaled score

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]					
1 [MASK]					
2 [SEP]					
3 cat					
4 [SEP]					

row total

0 [CLS]	
1 [MASK]	
2 [SEP]	
3 cat	
4 [SEP]	

A

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]					
1 [MASK]					
2 [SEP]					
3 cat					
4 [SEP]					

Hint. $e^0 = 1$, $e^{0.5} \approx 1.65$, $e^1 \approx 2.72$, $e^{-1} \approx 0.37$. Divide each entry by its row total. Check: every row of A adds up to 1.

Step 10: Head 1: blend the values, A · V

`head output = A · V · (5 × 5) · (5 × 2) = 5 × 2`

Head 1 output

	0	1
0 [CLS]		
1 [MASK]		
2 [SEP]		
3 cat		
4 [SEP]		

Hint. out[0 [CLS], 0] = A[0 [CLS] → 0]×V[0, 0] + A[0 [CLS] → 1]×V[1, 0] + A[0 [CLS] → 2]×V[2, 0] + A[0 [CLS] → 3]×V[3, 0] + A[0 [CLS] → 4]×V[4, 0].

Step 11: Head 2: queries, keys and values

`Q = X0 · W_Q, K = X0 · W_K, V = X0 · W_V · (5 × 4) · (4 × 2) = 5 × 2`

W_Q

	0	1
d0	0	1
d1	0	0
d2	0	0
d3	0	1

W_K

	0	1
d0	-1	0
d1	1	-1
d2	0	-1
d3	0	-1

W_V

	0	1
d0	1	0
d1	0	0
d2	0	0
d3	1	0

Q

	0	1
0 [CLS]		
1 [MASK]		
2 [SEP]		
3 cat		
4 [SEP]		

K

	0	1
0 [CLS]		
1 [MASK]		
2 [SEP]		
3 cat		
4 [SEP]		

V

	0	1
0 [CLS]		
1 [MASK]		
2 [SEP]		
3 cat		
4 [SEP]		

Hint. Each entry is one row of X0 dotted with one column of the weights: Q[0 [CLS], 0] = X0[0 [CLS], d0]×0 + X0[0 [CLS], d1]×0 + X0[0 [CLS], d2]×0 + X0[0 [CLS], d3]×0. The real model also adds a bias here; this sheet leaves it out.

Step 12: Head 2: scores S = Q · K^T

`S[r → c] = Q[r] · K[c] · shape 5 × 5`

S (rows: query, columns: key)

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]					
1 [MASK]					
2 [SEP]					
3 cat					
4 [SEP]					

Hint. S[0 [CLS] → 1 [MASK]] = Q[0 [CLS], 0]×K[1 [MASK], 0] + Q[0 [CLS], 1]×K[1 [MASK], 1]. No mask: every piece may look at every other, in both sentences, in both directions.

Step 13: Head 2: scale by √d_k

`S ÷ √2 = S ÷ 1.41`

S ÷ 1.41

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]					
1 [MASK]					
2 [SEP]					
3 cat					
4 [SEP]					

Hint. Divide every score by 1.41.

Step 14: Head 2: softmax, row by row

`A[r, c] = e^x[r, c] ÷ (sum of e^x across row r)`

e^x of each scaled score

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]					
1 [MASK]					
2 [SEP]					
3 cat					
4 [SEP]					

row total

0 [CLS]	
1 [MASK]	
2 [SEP]	
3 cat	
4 [SEP]	

A

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]					
1 [MASK]					
2 [SEP]					
3 cat					
4 [SEP]					

Hint. $e^0 = 1$, $e^{0.5} \approx 1.65$, $e^1 \approx 2.72$, $e^{-1} \approx 0.37$. Divide each entry by its row total. Check: every row of A adds up to 1.

Step 15: Head 2: blend the values, A · V

`head output = A · V · (5 × 5) · (5 × 2) = 5 × 2`

Head 2 output

	0	1
0 [CLS]		
1 [MASK]		
2 [SEP]		
3 cat		
4 [SEP]		

Hint. out[0 [CLS], 0] = A[0 [CLS] → 0]×V[0, 0] + A[0 [CLS] → 1]×V[1, 0] + A[0 [CLS] → 2]×V[2, 0] + A[0 [CLS] → 3]×V[3, 0] + A[0 [CLS] → 4]×V[4, 0].

Step 16: Combine the heads with W_O

`MHA = Concat(heads) · W_O · (5 × 4) · (4 × 4) = 5 × 4`

Concat

	h1·0	h1·1	h2·0	h2·1
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

W_O

	d0	d1	d2	d3
d0	-1	1	0	0
d1	0	0	1	0
d2	0	1	0	1
d3	0	0	-1	0

MHA

	d0	d1	d2	d3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

Hint. Concat: place the head outputs side by side, head 1 first. Then MHA[0 [CLS], d0] = Concat[0 [CLS], 0]×(−1) + Concat[0 [CLS], 1]×0 + Concat[0 [CLS], 2]×0 + Concat[0 [CLS], 3]×0.

Step 17: Add & Norm (1)

`R1 = X0 + MHA, then X1 = (R1 - row mean) ÷ row sd`

R1 = X0 + MHA

	d0	d1	d2	d3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

mean

0 [CLS]	
1 [MASK]	
2 [SEP]	
3 cat	
4 [SEP]	

sd

0 [CLS]	
1 [MASK]	
2 [SEP]	
3 cat	
4 [SEP]	

X1

	d0	d1	d2	d3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

Hint. Row mean = (sum of the 4 numbers) ÷ 4. Row sd = √(average of (value − mean)²), dividing by 4. Then (value − row mean) ÷ row sd. γ = 1 and β = 0 here, so nothing else changes. Check: each row adds up to about 0.

Step 18: Feed-forward: expand, then GELU

`P = X1 · W1 + b1 · (5 × 4) · (4 × 4) = 5 × 4, then I = GELU(P)`

W1

	i0	i1	i2	i3
d0	1	1	0	-1
d1	-1	0	0	0
d2	-1	-1	0	1
d3	1	-1	0	0

b1

i0	i1	i2	i3
0	1	-1	1

P = X1 · W1 + b1

	i0	i1	i2	i3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

I = GELU(P)

	i0	i1	i2	i3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

Hint. P[0 [CLS], i0] = X1[0 [CLS], d0]×1 + X1[0 [CLS], d1]×(−1) + X1[0 [CLS], d2]×(−1) + X1[0 [CLS], d3]×1 + 0. BERT swaps the Transformer's ReLU for GELU, a smooth version: GELU(x) = 0.5 · x · (1 + tanh(0.7979 · (x + 0.0447 · x³))). Reference values: GELU(−2) ≈ −0.05, GELU(−1) ≈ −0.16, GELU(−0.5) ≈ −0.15, GELU(0) = 0, GELU(0.5) ≈ 0.35, GELU(1) ≈ 0.84, GELU(2) ≈ 1.95. Big positive numbers pass almost unchanged and big negative ones become almost 0.

Step 19: Feed-forward: compress back

`F = I · W2 + b2 · (5 × 4) · (4 × 4) = 5 × 4`

W2

	d0	d1	d2	d3
i0	0	-1	0	1
i1	0	0	0	1
i2	0	0	0	0
i3	0	0	0	1

b2

d0	d1	d2	d3
1	0	0	-1

F

	d0	d1	d2	d3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

Hint. F[0 [CLS], d0] = I[0 [CLS], i0]×0 + I[0 [CLS], i1]×0 + I[0 [CLS], i2]×0 + I[0 [CLS], i3]×0 + 1. Unlike ReLU, GELU leaves small negative numbers, so no term drops out for free.

Step 20: Add & Norm (2): the encoder output T

`R2 = X1 + F, then T = (R2 - row mean) ÷ row sd`

R2 = X1 + F

	d0	d1	d2	d3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

mean

0 [CLS]	
1 [MASK]	
2 [SEP]	
3 cat	
4 [SEP]	

sd

0 [CLS]	
1 [MASK]	
2 [SEP]	
3 cat	
4 [SEP]	

T (5 × 4)

	d0	d1	d2	d3
0 [CLS]				
1 [MASK]				
2 [SEP]				
3 cat				
4 [SEP]				

Hint. The same recipe as Add & Norm (1). Row 0 of T is C, the [CLS] vector; row 1 is the vector at the masked position.

The encoder is done: one context vector per piece. Now the two training heads read just two rows of T.

Step 21: MLM head: transform the masked row

`t = T[1] · pre = t · W_t + b_t · g = GELU(pre) · u = LayerNorm(g)`

W_t

	d0	d1	d2	d3
d0	0	1	0	0
d1	-1	1	-1	0
d2	-1	0	1	0
d3	1	0	1	0

b_t

d0	d1	d2	d3
0	0	0	0

t = T row 1

d0	d1	d2	d3

pre

d0	d1	d2	d3

g = GELU(pre)

d0	d1	d2	d3

mean

sd

u

d0	d1	d2	d3

Hint. Only the masked position goes through this head; the other rows of T are ignored. pre[d0] = t[d0]×0 + t[d1]×(−1) + t[d2]×(−1) + t[d3]×1 + 0. GELU as before, then one LayerNorm over the 4 numbers.

Step 22: MLM head: a score for every piece

`logit[v] = u · (token-table row v) · shape 1 × 7, then softmax · loss = -ln P[“red”]`

logits

[PAD]	[CLS]	[SEP]	[MASK]	cat	red	runs

e^x

[PAD]	[CLS]	[SEP]	[MASK]	cat	red	runs

total

P

[PAD]	[CLS]	[SEP]	[MASK]	cat	red	runs

MLM loss = –ln P[red]

top piece

Hint. The output layer reuses the token table from step 1, read row by row: logit[cat] = u · row cat. The real model adds an output bias; here it is 0. The loss only looks at the true word “red”: the more probability it got, the smaller the loss.

Step 23: NSP head: pool [CLS] and score the pair

`c = T[0] · pooled = tanh(c · W_p + b_p) · logits = pooled · W_nᵀ + b_n · loss = -ln P[label]`

W_p

	d0	d1	d2	d3
d0	-1	0	1	-1
d1	-1	0	0	0
d2	0	0	0	-1
d3	0	-1	-1	1

b_p

d0	d1	d2	d3
-1	1	0	-1

W_n (one row per class)

	d0	d1	d2	d3
IsNext	0	-1	0	-1
NotNext	0	-1	-1	-1

b_n

IsNext	NotNext
-1	0

c = T row 0

d0	d1	d2	d3

c · W_p + b_p

d0	d1	d2	d3

pooled = tanh(…)

d0	d1	d2	d3

logits

IsNext	NotNext

e^x

IsNext	NotNext

total

P

IsNext	NotNext

NSP loss = −ln P[NotNext]

> Hint. tanh squashes each number into −1 to 1 (tanh 0 = 0, tanh 1 ≈ 0.76, tanh −1 ≈ −0.76). logit[IsNext] = pooled · the IsNext row of W_n + (−1).

Step 24: Add the two losses

`total loss = MLM loss + NSP loss`

total loss

> Hint. This is the one number that is added: two losses, during pre-training, so one backward pass trains both heads and the shared encoder. At inference nothing is added. To fill in the blank you only read the MLM probabilities (your "top piece" above), and for a real task you drop both heads and fine-tune a new small one on C or T.

You pushed "[CLS] [MASK] [SEP] cat [SEP]" through a BERT encoder layer and both pre-training heads by hand. The weights are random, so the top piece is a guess, not "red". Training is what nudges every table you used until the guess is right.

Answer key · worksheet #38181

Worked with every value rounded to 2 decimals, the same way you did. If you only differ in the last digit, that is rounding.

input IDs

[CLS]	[MASK]	[SEP]	cat	[SEP]
1	3	2	4	2

segment IDs

[CLS]	[MASK]	[SEP]	cat	[SEP]
0	0	0	1	1

E_tok

	d0	d1	d2	d3
0 [CLS]	-1	-1	0	0
1 [MASK]	1	1	-1	1
2 [SEP]	-1	-1	-1	-1
3 cat	1	1	0	1
4 [SEP]	-1	-1	-1	-1

E_seg

	d0	d1	d2	d3
0 [CLS]	-1	0	-1	-1
1 [MASK]	-1	0	-1	-1
2 [SEP]	-1	0	-1	-1
3 cat	0	1	-1	-1
4 [SEP]	0	1	-1	-1

E_pos

	d0	d1	d2	d3
0 [CLS]	0	0	1	-1
1 [MASK]	-1	1	-1	1
2 [SEP]	1	-1	0	-1
3 cat	-1	0	1	1
4 [SEP]	-1	1	0	0

X_sum

	d0	d1	d2	d3
0 [CLS]	-2	-1	0	-2
1 [MASK]	-1	2	-3	1
2 [SEP]	-1	-2	-2	-3
3 cat	0	2	0	1
4 [SEP]	-2	1	-2	-2

mean

0 [CLS]	-1.25
1 [MASK]	-0.25
2 [SEP]	-2
3 cat	0.75
4 [SEP]	-1.25

sd

0 [CLS]	0.83
1 [MASK]	1.92
2 [SEP]	0.71
3 cat	0.83
4 [SEP]	1.3

X0

	d0	d1	d2	d3
0 [CLS]	-0.9	0.3	1.51	-0.9
1 [MASK]	-0.39	1.17	-1.43	0.65
2 [SEP]	1.41	0	0	-1.41
3 cat	-0.9	1.51	-0.9	0.3
4 [SEP]	-0.58	1.73	-0.58	-0.58

head 1 Q

	0	1
0 [CLS]	-0.3	0.9
1 [MASK]	-1.17	-0.65
2 [SEP]	0	1.41
3 cat	-1.51	-0.3
4 [SEP]	-1.73	0.58

head 1 K

	0	1
0 [CLS]	-1.2	-0.3
1 [MASK]	-1.56	-1.17
2 [SEP]	1.41	0
3 cat	-2.41	-1.51
4 [SEP]	-2.31	-1.73

head 1 V

	0	1
0 [CLS]	0.61	-0.3
1 [MASK]	-1.82	-1.17
2 [SEP]	1.41	0
3 cat	-1.8	-1.51
4 [SEP]	-1.16	-1.73

head 1 S

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]	0.09	-0.58	-0.42	-0.64	-0.86
1 [MASK]	1.6	2.59	-1.65	3.8	3.83
2 [SEP]	-0.42	-1.65	0	-2.13	-2.44
3 cat	1.9	2.71	-2.13	4.09	4.01
4 [SEP]	1.9	2.02	-2.44	3.29	2.99

head 1 S ÷ 1.41

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]	0.06	-0.41	-0.3	-0.45	-0.61
1 [MASK]	1.13	1.84	-1.17	2.7	2.72
2 [SEP]	-0.3	-1.17	0	-1.51	-1.73
3 cat	1.35	1.92	-1.51	2.9	2.84
4 [SEP]	1.35	1.43	-1.73	2.33	2.12

head 1 e^x

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]	1.06	0.66	0.74	0.64	0.54
1 [MASK]	3.1	6.3	0.31	14.88	15.18
2 [SEP]	0.74	0.31	1	0.22	0.18
3 cat	3.86	6.82	0.22	18.17	17.12
4 [SEP]	3.86	4.18	0.18	10.28	8.33

total

0 [CLS]	3.64
1 [MASK]	39.77
2 [SEP]	2.45
3 cat	46.19
4 [SEP]	26.83

head 1 A

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]	0.29	0.18	0.2	0.18	0.15
1 [MASK]	0.08	0.16	0.01	0.37	0.38
2 [SEP]	0.3	0.13	0.41	0.09	0.07
3 cat	0.08	0.15	0	0.39	0.37
4 [SEP]	0.14	0.16	0.01	0.38	0.31

head 1 output

	0	1
0 [CLS]	-0.37	-0.83
1 [MASK]	-1.34	-1.43
2 [SEP]	0.28	-0.5
3 cat	-1.36	-1.43
4 [SEP]	-1.24	-1.34

head 2 Q

	0	1
0 [CLS]	0	-1.8
1 [MASK]	0	0.26
2 [SEP]	0	0
3 cat	0	-0.6
4 [SEP]	0	-1.16

head 2 K

	0	1
0 [CLS]	1.2	-0.91
1 [MASK]	1.56	-0.39
2 [SEP]	-1.41	1.41
3 cat	2.41	-0.91
4 [SEP]	2.31	-0.57

head 2 V

	0	1
0 [CLS]	-1.8	0
1 [MASK]	0.26	0
2 [SEP]	0	0
3 cat	-0.6	0
4 [SEP]	-1.16	0

head 2 S

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]	1.64	0.7	-2.54	1.64	1.03
1 [MASK]	-0.24	-0.1	0.37	-0.24	-0.15
2 [SEP]	0	0	0	0	0
3 cat	0.55	0.23	-0.85	0.55	0.34
4 [SEP]	1.06	0.45	-1.64	1.06	0.66

head 2 S ÷ 1.41

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]	1.16	0.5	-1.8	1.16	0.73
1 [MASK]	-0.17	-0.07	0.26	-0.17	-0.11
2 [SEP]	0	0	0	0	0
3 cat	0.39	0.16	-0.6	0.39	0.24
4 [SEP]	0.75	0.32	-1.16	0.75	0.47

head 2 e^x

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]	3.19	1.65	0.17	3.19	2.08
1 [MASK]	0.84	0.93	1.3	0.84	0.9
2 [SEP]	1	1	1	1	1
3 cat	1.48	1.17	0.55	1.48	1.27
4 [SEP]	2.12	1.38	0.31	2.12	1.6

total

0 [CLS]	10.28
1 [MASK]	4.81
2 [SEP]	5
3 cat	5.95
4 [SEP]	7.53

head 2 A

	[CLS]	[MASK]	[SEP]	cat	[SEP]
0 [CLS]	0.31	0.16	0.02	0.31	0.2
1 [MASK]	0.17	0.19	0.27	0.17	0.19
2 [SEP]	0.2	0.2	0.2	0.2	0.2
3 cat	0.25	0.2	0.09	0.25	0.21
4 [SEP]	0.28	0.18	0.04	0.28	0.21

head 2 output

	0	1
0 [CLS]	-0.93	0
1 [MASK]	-0.58	0
2 [SEP]	-0.66	0
3 cat	-0.79	0
4 [SEP]	-0.87	0

Concat

	h1·0	h1·1	h2·0	h2·1
0 [CLS]	-0.37	-0.83	-0.93	0
1 [MASK]	-1.34	-1.43	-0.58	0
2 [SEP]	0.28	-0.5	-0.66	0
3 cat	-1.36	-1.43	-0.79	0
4 [SEP]	-1.24	-1.34	-0.87	0

MHA

	d0	d1	d2	d3
0 [CLS]	0.37	-1.3	-0.83	-0.93
1 [MASK]	1.34	-1.92	-1.43	-0.58
2 [SEP]	-0.28	-0.38	-0.5	-0.66
3 cat	1.36	-2.15	-1.43	-0.79
4 [SEP]	1.24	-2.11	-1.34	-0.87

R1

	d0	d1	d2	d3
0 [CLS]	-0.53	-1	0.68	-1.83
1 [MASK]	0.95	-0.75	-2.86	0.07
2 [SEP]	1.13	-0.38	-0.5	-2.07
3 cat	0.46	-0.64	-2.33	-0.49
4 [SEP]	0.66	-0.38	-1.92	-1.45

mean

0 [CLS]	-0.67
1 [MASK]	-0.65
2 [SEP]	-0.45
3 cat	-0.75
4 [SEP]	-0.77

sd

0 [CLS]	0.91
1 [MASK]	1.41
2 [SEP]	1.13
3 cat	1.01
4 [SEP]	1

X1

	d0	d1	d2	d3
0 [CLS]	0.15	-0.36	1.48	-1.27
1 [MASK]	1.13	-0.07	-1.57	0.51
2 [SEP]	1.4	0.06	-0.04	-1.43
3 cat	1.2	0.11	-1.56	0.26
4 [SEP]	1.43	0.39	-1.15	-0.68

P

	i0	i1	i2	i3
0 [CLS]	-2.24	0.94	-1	2.33
1 [MASK]	3.28	3.19	-1	-1.7
2 [SEP]	-0.05	3.87	-1	-0.44
3 cat	2.91	3.5	-1	-1.76
4 [SEP]	1.51	4.26	-1	-1.58

I = GELU(P)

	i0	i1	i2	i3
0 [CLS]	-0.03	0.78	-0.16	2.31
1 [MASK]	3.28	3.19	-0.16	-0.08
2 [SEP]	-0.02	3.87	-0.16	-0.15
3 cat	2.91	3.5	-0.16	-0.07
4 [SEP]	1.41	4.26	-0.16	-0.09

F

	d0	d1	d2	d3
0 [CLS]	1	0.03	0	2.06
1 [MASK]	1	-3.28	0	5.39
2 [SEP]	1	0.02	0	2.7
3 cat	1	-2.91	0	5.34
4 [SEP]	1	-1.41	0	4.58

R2

	d0	d1	d2	d3
0 [CLS]	1.15	-0.33	1.48	0.79
1 [MASK]	2.13	-3.35	-1.57	5.9
2 [SEP]	2.4	0.08	-0.04	1.27
3 cat	2.2	-2.8	-1.56	5.6
4 [SEP]	2.43	-1.02	-1.15	3.9

mean

0 [CLS]	0.77
1 [MASK]	0.78
2 [SEP]	0.93
3 cat	0.86
4 [SEP]	1.04

sd

0 [CLS]	0.68
1 [MASK]	3.56
2 [SEP]	0.99
3 cat	3.3
4 [SEP]	2.19

T

	d0	d1	d2	d3
0 [CLS]	0.56	-1.62	1.04	0.03
1 [MASK]	0.38	-1.16	-0.66	1.44
2 [SEP]	1.48	-0.86	-0.98	0.34
3 cat	0.41	-1.11	-0.73	1.44
4 [SEP]	0.63	-0.94	-1	1.31

t

d0	d1	d2	d3
0.38	-1.16	-0.66	1.44

pre

d0	d1	d2	d3
3.26	-0.78	1.94	0

g

d0	d1	d2	d3
3.26	-0.17	1.89	0

mean, sd

mean	sd
1.24	1.42

u

d0	d1	d2	d3
1.42	-0.99	0.46	-0.87

MLM logits

[PAD]	[CLS]	[SEP]	[MASK]	cat	red	runs
-0.58	-0.43	-0.02	-0.9	-0.44	0.87	-1.3

e^x

[PAD]	[CLS]	[SEP]	[MASK]	cat	red	runs
0.56	0.65	0.98	0.41	0.64	2.39	0.27

total

5.9

P

[PAD]	[CLS]	[SEP]	[MASK]	cat	red	runs
0.09	0.11	0.17	0.07	0.11	0.41	0.05

MLM loss, top piece

loss	top
0.89	red

c

d0	d1	d2	d3
0.56	-1.62	1.04	0.03

c · W_p + b_p

d0	d1	d2	d3
0.06	0.97	0.53	-2.57

pooled

d0	d1	d2	d3
0.06	0.75	0.49	-0.99

NSP logits

IsNext	NotNext
-0.76	-0.25

e^x

IsNext	NotNext
0.47	0.78

total

1.25

P

IsNext	NotNext
0.38	0.62

NSP loss

0.48

total loss

1.37